Pudica Algorithm
Group 1

Implementation Report

Yeredla Koushik Reddy, Taes Padhihary

Link to paper: [Pudica: Toward Near-Zero Queuing Delay in Congestion Control for Cloud Gaming]

Cloud gaming is hard. For a good user experience, delay needs to stay <= 50ms consistently. So, the queuing delay needs to converge to near zero.

The authors wrote the Pudica algorithm to do this. Pudica paces packets to measure the *Bandwidth Utilization Ratio (BUR)*. It uses this BUR to adjust the video bitrate and keep the queue empty.

We tried to implement the algorithms of the paper from scratch in C++. In this report, we highlight the details of the implementations in brief, and then analyze our evaluations of the algorithm.

The following is a short summary. Please find the detailed report here: https://github.com/toyess-37/pudica/blob/main/reports/report_detailed.pdf

1 Implementation Details

Architecture limitation

We always need the `receiver` to be on before we can proceed sending packets by sender.

More precisely, our usage of `sender` is: `./sender <target_ip> <port>` and `receiver` runs as `./receiver <port>`. So, the sender is a *client* here. It knows the receiver's ip address and fires packets there. But, in a real deployment, it is supposed to be the Controller running at the typical "server" side.

Since our focus is more on replicating the algorithm, we think we'll not be doing this correction as of now.

In whatever follows, sender does the computation and sends the frames and receiver plays the games.

1.1. Sender Architecture

The sender is implemented in `sender.cc` as a class called `PudicaSender`. It runs two threads in parallel: a **pacer thread** and a **listener thread**. Both threads share the current bitrate and pacing multiplier.

1.1.1. Shared Frame Table

The central data structure is a shared unordered map called `table`, protected by a single mutex and modified by both the pacer and listener.

```
unordered_map<uint32_t, Frame> table;
mutex table_mtx;
```

The pacer inserts a frame record before sending packets. Each entry `Frame` holds everything needed to reconstruct the frame's delay measurement:

```
struct Frame {
  uint64_t created_at = 0;  // local clock when pacer inserted
  uint64_t t0         = 0;  // echoed_send of IS_FIRST
  uint64_t t1_recv    = 0;  // recv_time   of IS_LAST
  vector<double> probes;    // computed Ti values
  uint32_t bytes_out  = 0;
  bool got_first = false; bool got_last  = false; bool done = false;
  bool complete() const { return got_first && got_last && probes.size() == N_PROBE; }
};
```

The sender also maintains two running counters: `running_inflight_frames` and `running_inflight_bytes`. These give the controller an O(1) view of in-flight load (otherwise we would have needed to hold the lock on table and do a scan every time). Two additional variables `last_done_fid` (highest frame successfully evaluated) and `oldest_inflight_id` (lowest frame still waiting in the table) track progress of the frames.

`last_done_fid` increases only when `evaluate()` succeeds. This is needed because the cleanup is tied to completed frames, not to whatever frame that happened to arrive the most recently.

1.1.2. Pacer Thread

The pacer thread runs a loop that fires once every frame interval. At each iteration it inserts a table entry, sends the frame data packets, then sends the probes.

Frame packets

The pacer calculates how many bytes belong to this frame from the current bitrate and splits them into 1400-byte chunks. Before the first `sendto()`, it acquires the table lock and inserts the frame record with `bytes_out` and `created_at` already set, and increments the running counters.

```
{
  lock_guard<mutex> lk(table_mtx);
  auto &fr      = table[fid];
  fr.bytes_out  = f_bytes; fr.created_at = now_microsecs();
  running_inflight_frames++; running_inflight_bytes += f_bytes;
}
```

Each payload packet is then sent with a small gap (`pkt_gap`) so that total transmission fits inside the **sensible period** (`INTERVAL / rho`). The first and last packets are flagged with `IS_FIRST` and `IS_LAST`.

```
double sensible  = INTERVAL / rho;      double pkt_gap   = sensible / pkts;
double agnostic  = INTERVAL - sensible; double probe_gap = agnostic / (N_PROBE + 1);

for (uint32_t pid = 0; pid < pkts && running; pid++) {
  // ... build hdr, set IS_FIRST / IS_LAST flags ...
  sendto(sock, buf, sizeof(buf), 0, (sockaddr*)&dest, dest_len);
  if (pid < pkts - 1) precise_sleep(pkt_gap);
}
```

Probe packets

After all frame packets are sent, the pacer sends `N_PROBE = 4` payload-free probe packets spread across the **agnostic period**.

Each probe is tagged with `IS_PROBE` and given a packet id of `UINT32_MAX - i` so it won't get confused with real data.

```
for (uint32_t i = 0; i < N_PROBE &&
running; i++) {
  precise_sleep(probe_gap);
  // ... build phdr with IS_PROBE flag ...
  sendto(sock, &phdr, ...);
}
```

Note on sleep

The sender uses a custom `precise_sleep()` function instead of `this_thread::sleep_for()`.

1.1.3. Listener Thread

The listener thread receives ACK packets sent back by the receiver, updates the shared table, and drives the controller. Its state consists of the OWD sliding window, `d_min`, `rtt_min`, `last_done_fid`, and `oldest_inflight_fid`.

Next: Receiver Architecture

OWD and D_min tracking

For every ACK, the listener computes the one-way delay (`owd = recv_time - send_time`) and maintains a sliding 10-second monotone-minimum deque. The front of the deque is always `d_min` .

```
while (!owd_window.empty() && (int64_t)owd_window.front().ts < cutoff) owd_window.pop_front();
while (!owd_window.empty() && owd_window.back().owd >= owd) owd_window.pop_back();
owd_window.push_back({now, owd});
d_min = owd_window.front().owd;
```

Table update

The listener looks up the arriving ACK's `fid` in the table. If the entry exists and is not yet done, it updates the relevant fields: `t0` on `IS_FIRST` , `t1_recv` on `IS_LAST` , and appends a T_i value on `IS_PROBE` .

```
auto it = table.find(fid);
if (it != table.end() && !it->second.done) {
  Frame &fr = it->second;
  if (ack->flags & IS_FIRST) { fr.t0 = ack->echoed_send; fr.got_first = true; }
  if (ack->flags & IS_LAST)  { fr.t1_recv = ack->recv_time; fr.got_last = true; }
  if (ack->flags & IS_PROBE) { fr.probes.push_back(min(min(raw_T, Hi), T_bound)); }
  if (fr.complete()) { // after receiving both IS_FIRST, IS_LAST and probes.size() == 4
    evaluate(fid, fr, recv_rate, d_min, rtt_min);
    running_inflight_frames--; running_inflight_bytes -= fr.bytes_out;
    table.erase(it);  // erase immediately
  }
}
```

Frames are erased from the table **immediately** after evaluation. Because `last_done_fid` only advances inside `evaluate()` , every completed frame is gone as soon as the controller has seen it.

Probe delay computation (T_i)

When a probe ACK arrives, the listener computes its corrected queuing delay using the three-way minimum from the paper's

$T_i = \min(D_i - D_{\min}, H_i, T_{\text{packet}})$

where:

- H_i is the gap between the frame's last packet arrival and the probe's arrival.
- T_{packet} is the inter-probe spacing.

```
double raw_T = max(0.0, D_pkt - Dmin);
double Hi = (recv_time_probe -
fr.t1_recv) / 1e6;

double T_bound = (1.0 - 1.0/rho) * L_SEC /
(N_PROBE + 1);
fr.probes.push_back(min(min(raw_T, Hi),
T_bound));
```

Next-delay fallback and timeout

After each table update, the listener checks the **oldest in-flight frame** by looking up `oldest_inflight_fid` . Two responses are possible depending on how long that frame has been waiting:

- **Fallback (age > 2 frame intervals).** If the oldest frame has been in flight longer than `NEXT_DELAY_THRESH` , the listener calls `on_inflight_age()` , which applies a one-frame 15% bitrate reduction and sets `restore_next` so the controller reverts at the next ack call.

```
auto fallback
= ctrl.on_inflight_age(age);
if (fallback.has_value()) {
  bitrate.store(fallback->bitrate);
  pacing.store(fallback->pacing);
}
```


Next: Receiver Architecture

- **Timeout (age > 10 frame intervals).** If the oldest frame exceeds `TIMEOUT`, the listener concludes that the network has dropped enough packets that the frame will never complete. It calls `on_frame_loss()`, decrements the counters, erases the frame, and increases `oldest_inflight_fid`.

```
if (age > PudicaAlgorithm::TIMEOUT) {
  ctrl.on_frame_loss();
  running_inflight_frames--;
  running_inflight_bytes -= oldest_it-
>second.bytes_out;
  table.erase(oldest_it);
  oldest_inflight_fid++;
}
```

After either path, `oldest_inflight_fid` is checked and increased past any frames that have already been completed and erased from the table:

```
while (table.find(oldest_inflight_fid) == table.end() && oldest_inflight_fid <= last_done_fid)
  oldest_inflight_fid++;
```

1.1.4. When is a Frame Considered Complete?

Discussion on completion of a frame

A frame is evaluated as soon as its first-packet ACK, last-packet ACK, and all four probe ACKs arrive. In well-behaved networks this fires naturally.

In environments with packet loss, probe packets are occasionally dropped and the frame never becomes complete on its own. This is then handled by the timeout path described above.

When `on_frame_loss()` is called for a timed-out frame, `evaluate()` is not called (since `D` may be undefined), and the controller instead forces a drain-mode bitrate cut directly.

The paper doesn't discuss much about this. It assumes the ideal situation that we always have all the data as discussed in *Section 4.1, Paragraph: "Competing flow detection with probe packets"*.

1.2. Receiver Architecture

The receiver is a single-threaded UDP listener in `receiver.cc`. It sends back the timing information as UDP acknowledgement packets.

For every incoming packet, the receiver:

- records the arrival timestamp (`recv_ts`);
- builds a `RecvACK` struct that echoes the sender's original `send_time`, the local `recv_time`, the current receive rate, and the packet's `frame_id`, `packet_id`, and `flags`;
- sends the ACK back immediately via `sendto()`.

```
RecvACK ack{};
ack.echoed_send = hdr->send_time; // echoed
back sender time
ack.recv_time   = recv_ts;
ack.rate        = recv_rate;
ack.frame_id    = hdr->frame_id;
ack.flags       = hdr->flags; // IS_FIRST,
IS_LAST or IS_PROBE
sendto(sock, &ack, sizeof(RecvACK), ...);
```

The receive rate is updated every 100 ms:

```
bytes_acc += n;
if (elapsed >= 100000) {
  recv_rate = (8.0 * bytes_acc) / elapsed;
  bytes_acc = 0; last_calc = recv_ts;
}
```

Note on clocks

The receiver does **not** need clock sync with the sender. Since both `echoed_send` and `recv_time` carry the same clock offset, the subtraction in the sender cancels it out when computing the one-way delay.

Next: Implementation of the Algorithm

1.3. Implementation of the Algorithm

The algorithm is implemented over two files: `pudica_algo.h` (defines all constants, data structures, and the `Controller` class), and `pudica_algo.cc` (contains every computation; `sender.cc` feeds info).

1.3.1. Data Structures

These structs carry all the data that flows between the sender and the controller.

- `Sample` stores the corrected BUR and the encoding bitrate at the time that frame was sent. The controller keeps a sliding window of these.
- `FrameAck` is assembled by the listener thread once a frame is considered complete and passed to the controller. It contains every network measurement the algorithm needs.
- The controller returns the updated bitrate, pacing multiplier, and the corrected BUR of a frame in the `control_output` struct.

1.3.2. Different types of BUR Estimation

The raw BUR for a frame is computed as $R = (D - D_{\min})/L$ where L is the frame interval (16.67 ms at 60 fps).

The corrected BUR adds the probe-measured queuing delays from the agnostic period: $R^{\text{corr}} = R + \sum_{i=1}^{N_{\text{packet}}} \frac{T_i}{L}$. Each T_i is already bounded before being placed in `fr.probes`, so `corrected_BUR()` just sums them:

When `probes` is empty (a timed-out frame whose partial state was not evaluated), this function returns `raw_BUR` unchanged. The pacing multiplier $\rho = \gamma_p / \min(R^{\text{corr}}, 1)$ is updated immediately on every `on_frame_acked()` call, before any bitrate decision is made.

The smoothed BUR $\tilde{R}$ is a weighted average of the history: $\tilde{R} = \left(\sum_{k=1}^{N} \omega_k \cdot R_k \cdot \frac{B}{B_k}\right) / \left(\sum_{k=1}^{N} \omega_k\right)$ where all the weights are as discussed in the paper. Details omitted for brevity.

1.3.3. History Window

The paper specifies a time-based 200 ms sliding window:

```
history.push_back({bur_corr, current_bitrate, fa.now_microsecs});
while (!history.empty() &&
       (fa.now_microsecs - history.front().ts) > 200'000ULL)
  history.pop_front();
```

1.3.4. Bitrate Control

`on_frame_acked()` is a state machine with two top-level branches: congested ($R^{\text{corr}} > 1$) and not congested.

- **Congested branch:**

First congested frame (Temporary Fallback)

When `congested_frames == 1`, the controller saves the current bitrate and immediately reduces it by $\zeta = 0.15$ for one frame only. The `restore_next` flag ensures it reverts at the very start of the next `on_frame_acked()` call.

```
saved_rate      = current_bitrate;
restore_next    = true;
current_bitrate = max(current_bitrate *
(1.0 - ZETA), B_MIN);
```

Third consecutive congested frame (Active Drain)

When `congested_frames >= 3`, the controller enters drain mode. It calculates how fast in-flight data must be cleared within 200 ms, then sets the bitrate below the current receiving rate:

Next: Deviations from the paper

```
double drain_rate = (8.0 * fa.in_bytes) / (DRAIN_WIN * 1e6);
double new_rate   = ALPHA * fa.recv_rate - drain_rate;
current_bitrate   = max(new_rate, B_MIN);
adj_after         = fa.fid + fa.n_inflight;
```

`fa.in_bytes` comes directly from `running_inflight_bytes`. While draining, subsequent congested frames do nothing.

No action is taken on the second congested frame (`congested_frames == 2`) deliberately, to avoid triggering a full drain on what might be a delay spike.

- **Non-congested branch:**

When the corrected BUR falls back below 1.0, the branch checks two sub-cases.

If the controller was in drain mode, it exits immediately, restores the bitrate directly to the current receiving rate and clears the history.

```
draining = false;
current_bitrate = fa.recv_rate;
history.clear();
return {current_bitrate, current_pacing,
last_bur};
```

If the controller was not draining, it proceeds to the normal MI or AI-MD update, gated by `adj_after`. This gate ensures the controller waits for in-flight feedback before making another adjustment.

```
if (fa.fid > adj_after) {
  double smooth = smoothed_BUR(history,
current_bitrate);
  current_bitrate
= next_bitrate(current_bitrate, smooth,
frames_up);
  adj_after = fa.fid + fa.n_inflight;
}
```

1.3.5. Frame Loss Handling

When the listener times out a frame after `TIMEOUT = 10 * INTERVAL`, it calls `on_frame_loss()` on the controller instead of `on_frame_acked()`. This path skips BUR computation entirely (since D may be undefined) and forces the controller directly into drain mode with a hardcoded 50% bitrate cut.

This is a deliberate difference from the paper.

```
void Controller::on_frame_loss() {
  congested_frames = 3;
  draining         = true;
  restore_next     = false;
  saved_rate       = 0.0;
  current_bitrate  = max(current_bitrate *
0.5, B_MIN);
  history.clear();
}
```

1.3.6. MI and AI-MD

`next_bitrate()` implements both the efficiency and fairness phases using the smoothed BUR $\tilde{R}$.

When $\tilde{R} \leq \alpha = 0.85$, the controller is in the MI phase:

```
double xi = GAMMA_MI * ((1.0 + ALPHA) / (2.0 * safe_r) - 1.0);
new_rate  = current_rate * (1.0 + xi);
```

When $\alpha < \tilde{R} \leq 1$, the controller applies simultaneous AI and MD with adaptive step:

```
double tau = min(frames_up / 60.0, 5.0);
double I   = (B_MAX + pow(2.0, tau)/max(log(current_rate), B_MIN)) * (GAMMA_MD / 2.0);
double A   = I - GAMMA_MD * current_rate;
A = min(max(A, A_MIN), A_max * current_rate);
```

`frames_up` counts consecutive non-congested frames and resets every five seconds, preventing I from growing without bound.

1.3.7. Next-Delay Fallback

In addition to per-frame processing, the listener calls `on_inflight_age()` after every received ACK, passing the age of the oldest in-flight frame (`oldest_inflight_fid`). If a frame has been in flight for more than two frame intervals without completing, the controller reduces the bitrate by 15% for one frame:

```
optional<control_output> Controller::on_inflight_age(uint64_t age) {
  if (age < NEXT_DELAY_THRESH) return nullopt;
  if (restore_next || draining) return nullopt;
  saved_rate      = current_bitrate;
  restore_next    = true;
  current_bitrate = max(current_bitrate * (1.0 - ZETA), B_MIN);
  return control_output{current_bitrate, current_pacing, last_bur};
}
```

1.4. Deviations from the paper

1.4.1. Receiving Rate for Draining

Section 4.3, Paragraph: "Active Queue Draining"

The active queue-draining formula $B_{\text{new}} = \alpha \cdot r_{\text{recv}} - r_{\text{drain}}$ uses the *"average data receiving rate from the onset of congestion to the present moment"* as r_{recv}.

- **Our implementation:** We use instantaneous receiving rate of the latest ACK (`ack->rate`). We don't track inflight bytes since congestion and hence don't use the average rate. We adopted instantaneous rate for simplifying an already complicated implementation.
- **Effect:** In stable congestion this is a reasonable approximation. During rapid bandwidth drops, the instantaneous rate can be noisy.

1.4.2. B_{min} Lowered from 1 Mbps to 0.2 Mbps

The paper targets cloud gaming at bitrates in [1, 50] Mbps, with $B_{\text{min}} = 1$ Mbps.

- **Our implementation:** B_MIN = 0.2 Mbps. This was lowered to prevent the sender from stalling entirely on the low-bandwidth Mahimahi built-in LTE traces.

1.4.3. Bounds on various variables for safety

- Capped $\log(B)$ in the lower bound to `B_MIN` in the adaptive AI step. This prevents any possible divide by zero errors or overshoot in I.
- Hard coded `safe_r = max(bur_tilde, 0.01)` in the MI calculation. We did it to prevent division by near-zero quantity when BUR is very small. So, in our implementation, $\text{BUR} \geq 0.01$ always.

1.4.4. Summary of Deviations

#	Aspect	Paper	We
1	r_{recv} for drain/recovery	Average rate since congestion	instantaneous ACK rate
2	B_{min}	1 Mbps	0.2 Mbps
4	Bounded vars	Only Bitrate	Bitrate, $\log(B)$ and `safe_r`

Table 1. Deviations of our implementation from the Pudica paper

Next: Custom designed tests

2 Performance Evaluation

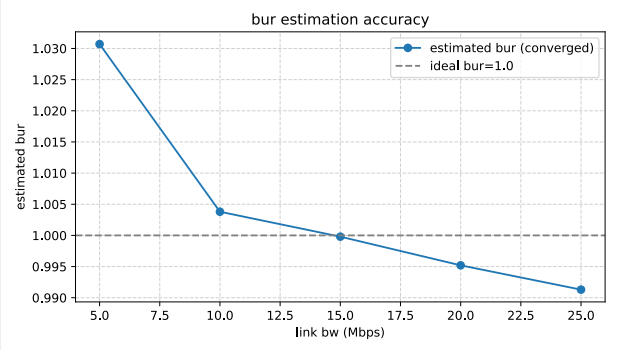
We evaluated the algorithm with different tests. The authors of the paper claim Pudica achieves “near-zero queuing delay” and high bandwidth use. We run multiple tests to evaluate these claims.

Setup details

- The testing was done in two Ubuntu-22.04LTS PCs in the lab over the same LAN. The tests involved emulating the network using the **Mahimahi** emulator.
- The paper displays *log-delays* at some places and delays at some places. We displayed the delays as they are, and smoothed them out using numpy’s convolve.
- We generated trace files for custom tests using Mahimahi and python scripts. The details of those files are excluded here. All the test files are available in the `evaluations` folder of the repository.
- For our testing, we used Python scripts for shell manipulation. The scripts are AI-generated, but have been cross-checked to be accurate. The file `utils.py` generates const traces of file type `.up` and defines how the plot etc. will go. Other files use variables from this file.

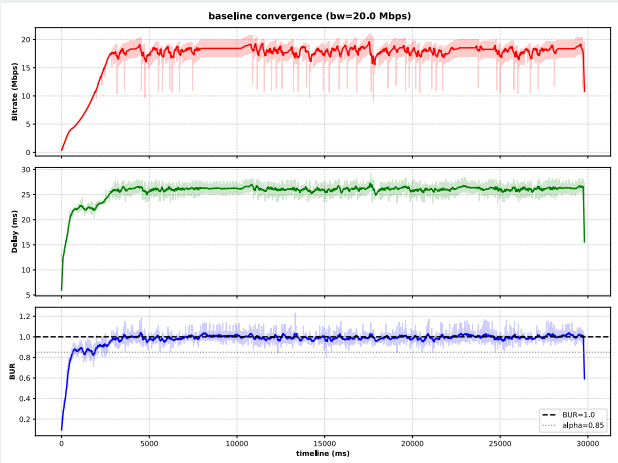
2.1. Custom designed tests

2.1.1. BUR Estimation Accuracy



We tested the BUR across different bandwidths (5 to 25 Mbps). The estimated BUR stays near 1. This means the core math of our implementation works very well.

2.1.2. Constant Link (20 Mbps)



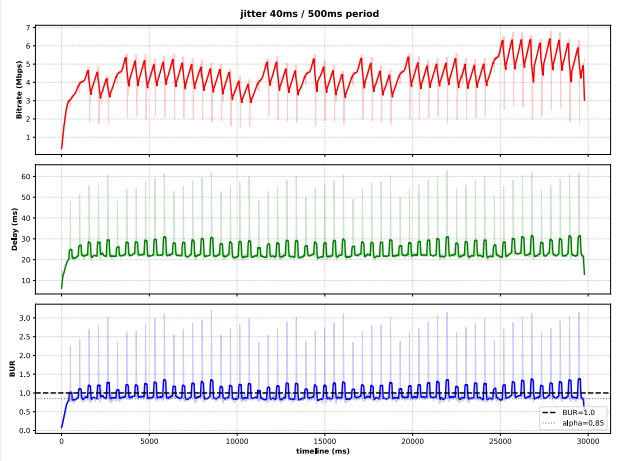
We gave Pudica a flat 20 Mbps link. It reaches around 18 Mbps fast. Delay stays low around 25ms. BUR stays near 1. This supports the paper’s claims. However, these are idealistic conditions and will not be there in the real internet.

2.1.3. Step Bandwidth Drop

We drop the link from 20 Mbps to 10 Mbps at the 10 seconds mark. Pudica drops the bitrate very fast. Thus spiking a sharp delay. The delay spikes up to almost 100ms instantly. However, the graph is close to what is also present in the paper.

Next: Mahimahi Default Tests [3G/4G]

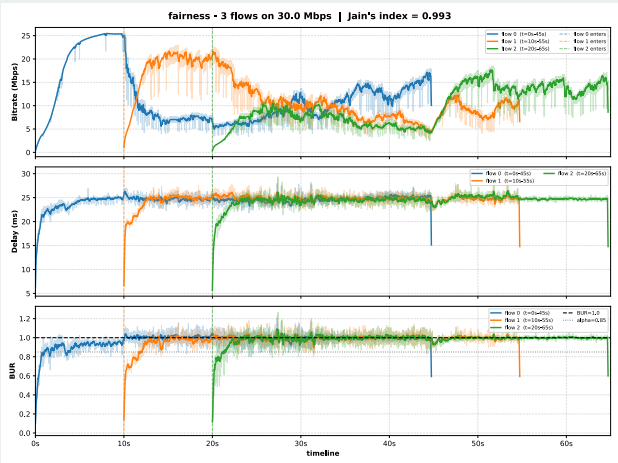
2.1.4. Jitter Test (40ms jitter / 500ms period)



This test shows a big problem. The paper claims in Appendix F (Figure 17) that Pudica handles jitter well. But our test shows Pudica panics. The bitrate behaves wildly and stays very low. The BUR also goes crazy.

We think the paper maybe shows a lucky run. **Our result shows Pudica is sensitive to jitter.** However, it still avoided any stall (by never letting bitrate go high).

2.1.5. Jain's Fairness Test

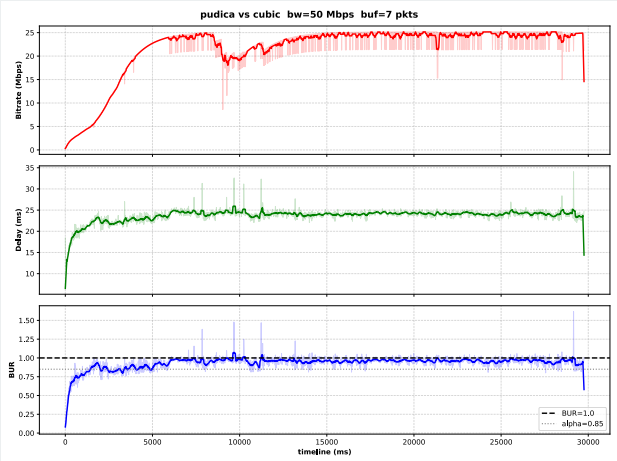


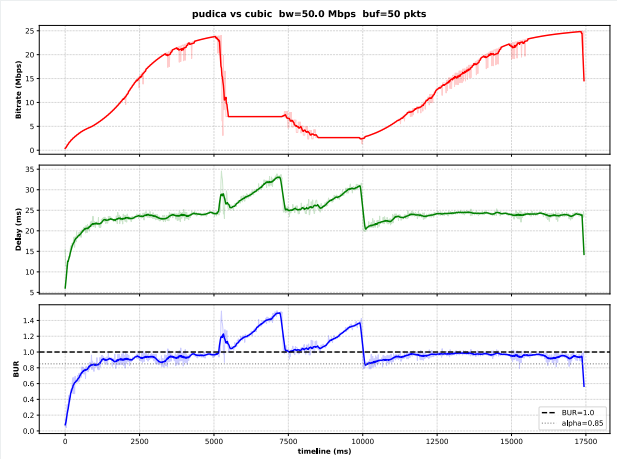
We start three Pudica flows on a 30Mbps link as described in the paper. The graph more or less resembles the paper's graphs. The Jain's fairness index is 0.993. The delay stays around 25ms. The paper claims great fairness, and our test agrees.

2.1.6. TCP Cubic Competition

We run Pudica against TCP Cubic. TCP Cubic is run from 5s to 10s as said in the paper.

- *With a small buffer (7 packets)*, Pudica dominates over TCP Cubic. The interesting point to note here is that Pudica doesn't fluctuate a lot when there are very less number of packets because "no buffer, no queuing delays".
- *With a large buffer (50 packets)*, Pudica drops its bitrate near 0 Mbps and lets Cubic take the bandwidth. Also the fluctuation and frequency in bandwidth adjust is quite observable.
- The paper says in Section 6 that Pudica "concedes" to buffer-filling flows (of course, only when there are any buffers to begin with like 50 packets instead of merely 7). But for shallow buffers, Pudica kills TCP Cubic.





2.2. Mahimahi Default Tests [3G/4G]

The results were pretty good. The test was conducted using the `tests_mm.py` file and the available built-in traces.

trace	avg_rate	avg_delay	p99_delay	stall_100ms%
ATT-LTE-driving-2016	3.26m	35.0ms	135.1ms	5.080%
ATT-LTE-driving	0.61m	60.8ms	133.7ms	9.590%
TMobile-LTE-driving	12.72m	26.3ms	66.2ms	0.250%
TMobile-LTE-short	2.29m	36.1ms	130.2ms	3.040%
TMobile-UMTS-driving	0.92m	78.2ms	134.4ms	38.610%
Verizon-EVDO-driving	1.39m	77.4ms	85.1ms	0.000%
Verizon-LTE-driving	4.42m	28.2ms	59.1ms	0.000%
Verizon-LTE-short	4.25m	28.7ms	108.5ms	1.390%

Figure 2. Mahimahi traces test

2.3. Zeus Tests [5G]

The results were pretty good. The test was conducted using the datasets available at the [ZEUS project](). We converted them to be usable for mahimahi and capped the maximum bitrate at 300Mbps.

trace	avg_rate	avg_delay	p99_delay	stall_100ms%
5GBeachStationary	21.78m	25.0ms	39.8ms	0.230%
5GBeachStationary2	24.05m	24.1ms	41.3ms	0.360%
5GBeachStationary3	23.73m	37.4ms	146.1ms	3.880%
5GCityDrive	19.46m	26.7ms	115.3ms	1.530%
5GCorniche	24.98m	23.2ms	24.7ms	0.000%
5GCornicheWalking	24.43m	23.3ms	24.8ms	0.000%
5GParkStationary1	12.91m	34.0ms	143.9ms	5.480%
5GParkStationary2	22.38m	24.0ms	67.9ms	0.340%

Figure 3. ZEUS 5G traces test